Architecture of Linux

Linux operating system

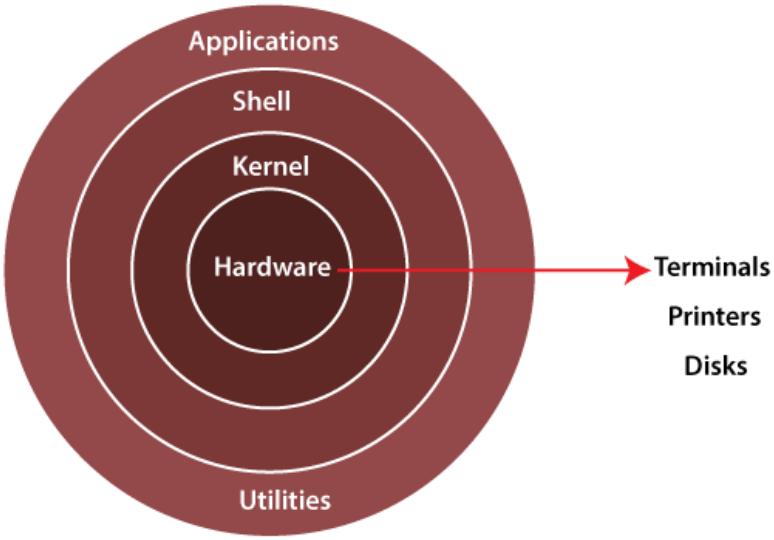
An operating system can be described as an interface among the computer hardware and the user of any computer. It is a group of software that handles the resources of the computer hardware and facilitates basic services for computer programs.

An operating system is an essential component of system software within a computer system. The primary aim of an operating system is to provide a platform where a user can run any program conveniently or efficiently.

Linux operating system history

In 1991, the Linux history started with the starting of a particular project by the Finland student **Linus Torvalds** for creating a new free **OS kernel**. The final Linux Kernel was remarked by continuous development throughout the history since then.

Architecture of Linux system



The Linux operating system's architecture mainly contains some of the components: **the Kernel, System Library, Hardware layer, System,** and **Shell utility**.

1. Kernel:- The kernel is one of the core section of an operating system. It is responsible for each of the major actions of the Linux OS. This operating system contains distinct types of modules and cooperates with underlying hardware directly. The kernel facilitates required abstraction for hiding details of low-level hardware or application programs to the system. There are some of the important kernel types which are mentioned below:

- Monolithic Kernel
- Micro kernels
- Exo kernels
- Hybrid kernels

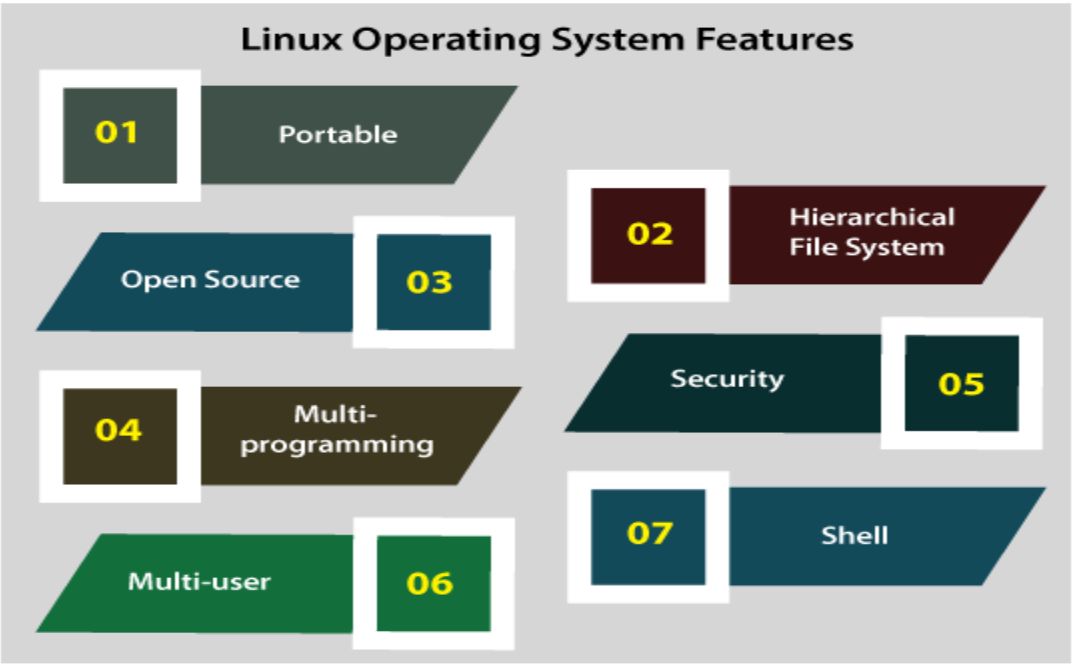
2. System Libraries:- These libraries can be specified as some special functions. These are applied for implementing the operating system's functionality and don't need code access rights of the modules of kernel.

3. System Utility Programs:- It is responsible for doing specialized level and individual activities.
4. Hardware layer:- Linux operating system contains a hardware layer that consists of several peripheral devices like CPU, HDD, and RAM.
5. Shell:- It is an interface among the kernel and user. It can afford the services of kernel. It can take commands through the user and runs the functions of the kernel. The shell is available in distinct types of OSes. These operating systems are categorized into two different types, which are the **graphical shells** and **command-line shells**.
The graphical line shells facilitate the graphical user interface, while the command line shells facilitate the command line interface. Thus, both of these shells implement operations. However, the graphical user interface shells work slower as compared to the command-line interface shells. There are a few types of these shells which are categorized as follows:

- Korn shell
- Bourne shell
- C shell
- POSIX shell

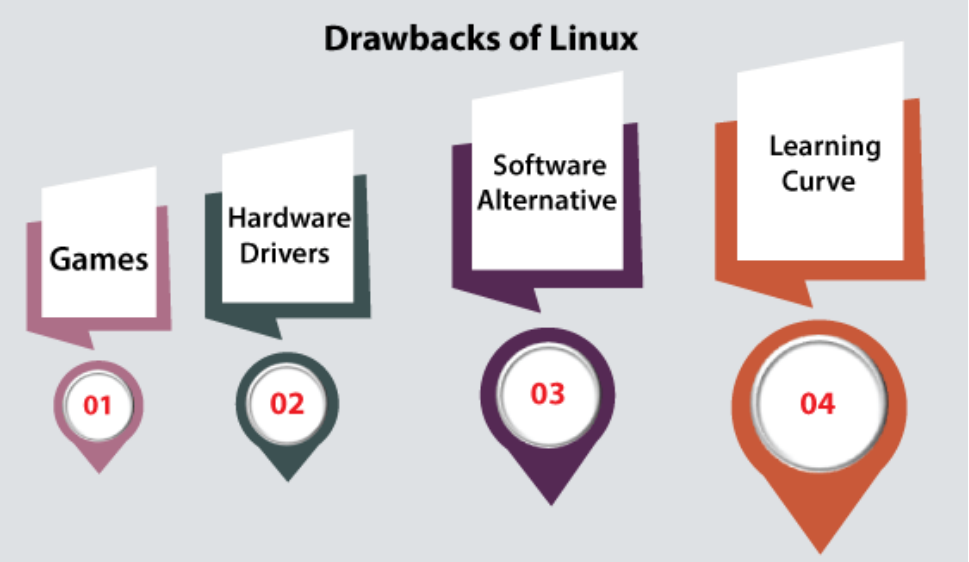
Linux Operating System Features

Some of the primary features of Linux OS are as follows:



- **Portable:** Linux OS can perform different types of hardware and the kernel of Linux supports the installation of any type of hardware environment.
- **Open source:** Linux operating system source code is available freely and for enhancing the capability of the Linux OS, several teams are performing in collaboration.
- **Multiprogramming:** Linux OS can be defined as a multiprogramming system. It means more than one application can be executed at the same time.
- **Multi-user:** Linux OS can also be defined as a multi-user system. It means more than one user can use the resources of the system such as **application programs, memory,** or **RAM** at the same time.
- **Hierarchical file system:** Linux OS affords a typical file structure where user files or system files are arranged.
- **Security:** Linux OS facilitates user security systems with the help of various features of authentication such as controlled access to specific files, password protection, or data encryption.

- **Shell:** Linux operating system facilitates a unique interpreter program. This type of program can be applied for executing commands of the operating system. It can be applied to perform various types of tasks such as call application programs and others.



- **Hardware drivers:** Most of the users of Linux face an issue while using Linux. Various companies of hardware prefer to build drivers for Mac or Windows due to they contain several users than Linux. Linux has small drivers for peripheral hardware than windows.
- **Software alternative:** Let's take the Photoshop example which is a famous tool for graphic editing. Photoshop exists for Windows; however, it is not available in Linux. Also, there are some other tools for photo editing but the Photoshop tool is more powerful as compare to others. Another example is MS office which is not present for Linux users.
- **Learning curve:** Linux isn't a very user-friendly operating system. Hence, it might be confusing for many beginners. Getting begun with Windows is efficient and easy for many beginners; however, understanding Linux working is complex. We have to understand the command line interface and finding for newer software is a little bit complex as well. When we face any issue in the OS, the searching solution is very problematic. Also, there are various experts for Mac and Windows as compare to Linux.
- **Games:** Several games are developed for Windows but unfortunately not for Linux. Because the platform of Windows is used widely. So, the developers of the games are more interested in windows.

Linux Operating System Applications

Linux is a billion-dollar corporation nowadays. Thousands of governments and companies are using Linux operating system across the world because of lower money, time, licensing fee, and affordability. Linux can be used within several types of electronic devices. These electronic devices are easily available for users worldwide

Linux Distribution

It is an OS that is composed of a software-based collection on Linux kernel or we can say the distribution includes the Linux Kernel. It is supporting software and libraries. We can obtain Linux-based OS by downloading any Linux distribution. These types of distributions exists for distinct types of devices such as **personal computers, embedded devices**, etc. Around more

than 600 Linux distributions are existed and a few of the famous Linux distributions are listed as follows:

- Deepin
- OpenSUSE
- Fedora
- Solus
- Debian
- Ubuntu
- Elementary
- Linux Mint
- Manjaro
- MX Linux

Are Ubuntu and Linux Differ?

The primary difference between window and Linux is that window is open source and free OS and its Linux distribution based on Debian, Whereas Linux is a large collection of open-source OSes that are working based on Linux kernel.

User mode vs Kernel mode

The code of kernel component runs in a unique privilege mood known as **kernel mode** along with complete access to every computer resource. This code illustrates an individual process, runs in an individual address space, and don't need the context switch. Hence, it is very fast and efficient.

Kernel executes all the processes and facilitates various services of a system to the processes. Also, it facilitates secured access to processes to hardware.

The support code that is not needed to execute in kernel mode is inside the system library. The user programs and other types of system programs are implemented in the user mode.
It includes no access to kernel mode and system hardware. User utilities/programs use the system libraries for accessing kernel functions to obtain low-level tasks of the system.

Features of Linux Operating System

LINUX is an open-source operating system used extensively worldwide. It is free and its source code is available for everyone under *GNU(General Public Licence)*.
Linux as an Operating System provides a wide range of features that can be used by a wide range of systems such as servers and computers.
Android *OS* is also based on the Linux kernel that extends its usage for building embedded systems and automation systems.

What are the Features of Linux Operating Systems?

The following are the features of the Linux Operating system:

1. Free and Open-Source

Linux is completely free of cost, and expenses are never a hindrance to using it as an operating system.
Linux is open-source. This means that modification of code, analysis of codes, redistribution of codes, or selling copies of the enhanced codes can be done by anyone in the world provided they come under the same license where the license also costs no charge.

Linux operating system is released under the ***GNU(General Public Licence)*** and is now one of the largest open-source projects worldwide.

2. Extremely Flexible

Linux has incorporated itself into embedded products like watches, digital equipment and supercomputing servers.

3. Lightweight Infrastructure

Linux consumes lesser storage space, and its installation requires around 4GB to 8GB of disk space.

Memory footprint or the amount of memory *(RAM)* used by the software while running is also less and it is compatible with all kinds of file formats like *text files, audio files, video files, graphic formats*, etc.

4. Graphical User Interface (GUI)

Even though *Linux* works on using the command line interface but it can be converted to be used like windows having a *Graphical user interface*. This is done mostly by installing packages. The most common way of having a *GUI* on the *Linux* environment is to log in to ***Ubuntu*** server and install its desktop environment.

5. End-to-end encryption

Linux allows end-to-end encryption while accessing data thus storing public keys in the server. All data is password protected and provides authentication to users. It also allows many security features and provides file permissions, a secure shell, etc.

6. Portable Environment

Linux works on any kind of environment and doesn't depend on the device being **high-ended** or **low-ended**. A large number of users can simultaneously use it anytime, any place, and on multiple devices. It supports all kinds of hardware to work on.

Multiple distributions or enterprises are also supported by *Linux*.

Linux has its own repository for software that can be used to install the required packages.

7. Shell/ Command-line Interface

The *Linux* command line interpreter is known as ***Shell*** that provides an interface between the user and kernel which then executes programs known as commands.

Hence, Linux uses *Command-line interface* to carry out the execution of tasks which is comparatively more efficient to execute and takes lesser time. It also takes lesser space in the memory,

8. Customized keyboard

Linux is available in all languages which makes it acceptable worldwide. That is why language can be changed from the keyboard according to the requirements and preferences.

Users can also build their own customized layout for keyboards.

9. Frequent New Updates

Linux operating system provides a wide range of accessible software updates that can be deployed and used according to requirements.

They update more frequently thus giving users an option to choose the updates and install them as per their need.

10. Hierarchical File System

Linux comes with a well-defined file structure where user files are arranged in a definite directory structure. According to the type of files, the directories are categorised as *Binary directories, configuration directories, Data directories, memory directories, Usr(Unix System Resources), var(variable directory)* and *non standard directories*.

11. Multi-user and Multi-programming

Linux allows multiple users to access the system resources at the same time and allows multiple applications to run at the same time.

The Linux command is a utility of the Linux operating system. All basic and advanced tasks can be done by executing commands. The commands are executed on the **Linux terminal**. The terminal is a command-line interface to interact with the system, which is similar to the command prompt in the Windows OS. *Commands in Linux are **case-sensitive**.*

Linux terminal is a user-friendly terminal as it provides various support options. To open the Linux terminal, press "**CTRL + ALT + T**" keys together, and execute a command by pressing the '**ENTER**' key.

The Linux command is a utility of the Linux operating system. All basic and advanced tasks can be done by executing commands. The commands are executed on the **Linux terminal**. The terminal is a command-line interface to interact with the system, which is similar to the command prompt in the Windows OS. *Commands in Linux are **case-sensitive**.*

Introduction to man

man is a built-in manual for utilizing Linux commands. It permits users to access the reference manual of a utility or a command to execute in the terminal. The man command contains the command description, examples, flags, applicable options, and other informative elements.

Syntax of man:

1. man [option(s)] keyword(s)

But generally [option(s)] are not used. Only keyword is written as an argument.

man Options

Commands	Function
man -aw	List all available sections of a command.
man -a	To view all man pages of a command.
sman -k (apropos)	Shows a list of results in man page containing a keyword match.
-f, whatis	It displays description from manual page if available.
whereis	Used to determine location of a man page
No Option	It shows the entire command manual.
Section-num	As the manual is categorized into two or more sections, the section-num option is applied to show only a specific manual section.
-w	This option is used to return the location where the manual page of a specified command is available.
-I	This option takes the command case-sensitive.

For example,

man ls

This command will display all the information about **'ls'** command as shown in the screen shot.

sudo command

Sudo is a command-line utility for Unix and Unix-based operating systems such as Linux and macOS. The utility provides an efficient way to temporarily grant users or user groups privileged access to system resources so that they can run commands that they cannot run under their regular accounts. Users can even be granted permissions to run commands under the root account -- the most powerful account on Unix-like systems. Sudo also logs all commands and arguments so that administrators can track the behavior of sudo users.

```
damon@Ubuntu20VirtualBox:~$ useradd test
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
damon@Ubuntu20VirtualBox:~$ sudo useradd test
[sudo] password for damon:
```

echo command

echo - display a line of text

Syntax

echo [SHORT-OPTION]... [STRING]...
echo LONG-OPTION

The echo is a bash built-in command that is used to echo the string(s) to the standard output. command that is used to echo the string(s) to the standard output.

Options

The program accepts the following options.

-n do not output the trailing newline
-e enable interpretation of backslash escapes
-E disable interpretation of backslash escapes (default)
-- help display this help and exit

--version
output version information and exit

If -e is in effect, the following sequences are recognized:

\\ backslash
\a alert (BEL)
\b backspace
\e escape
\f form feed
\n new line
\r carriage return
\t horizontal tab
\v vertical tab

Examples

echo a list of strings on command line.
$ echo "Shambhavi Vidya"
Output
Shambhavi Vidya

\n option creats a newline from where it is used.
$ echo -e "Shree \nLaxmi"
Output
Shree
Laxmi
Using the option \b inside the string removes the whitespace between the strings.
$ echo -e "Shree \bLaxmi"
ShreeLaxmi

Display output of a command in echo's argument. Use $(command) option of bash shell to show current date.
$ echo "The date is: $(date)"

The date is: Wed Mar 24 20:18:00 IST 2021
$ echo "The date is: $(date +%D)"
The date is: 03/24/21

printf command

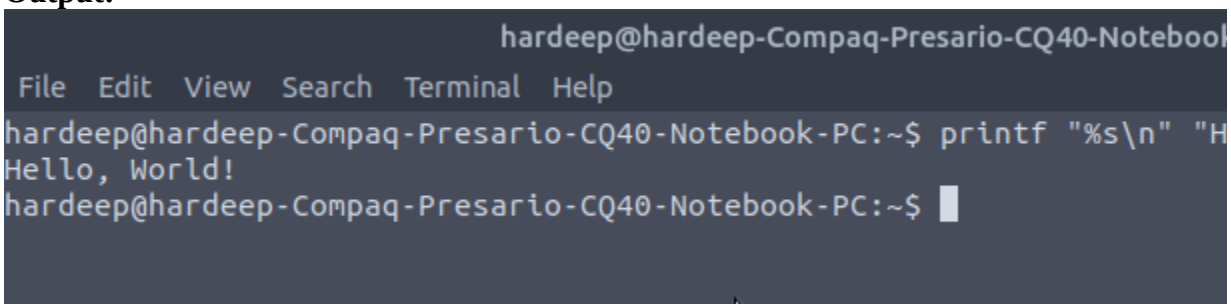
"*printf*" command in Linux is used to display the given string, number or any other format specifier on the terminal window. It works the same way as "printf" works in programming languages like C.

Syntax:
$printf [-v var] format [arguments]

Note: *printf* can have format specifiers, escape sequences or ordinary characters.

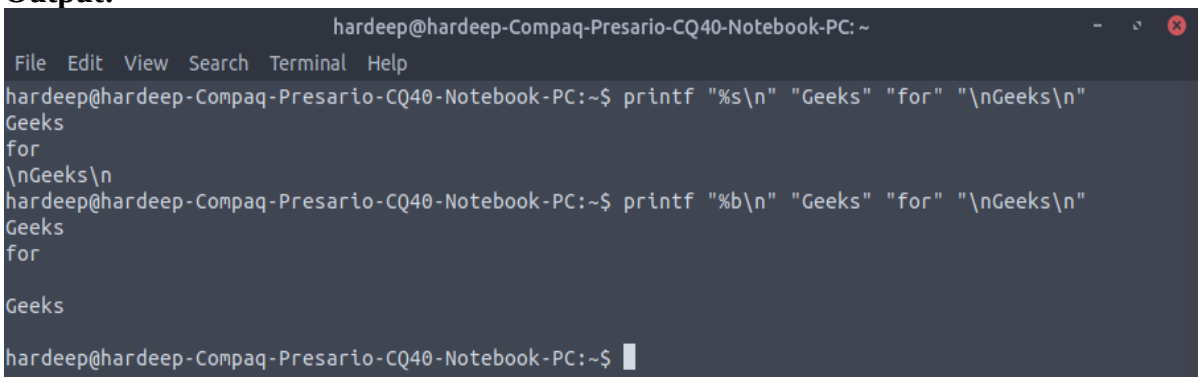
Format Specifiers: The most commonly used printf specifiers are %s, %b, %d, %x and %f.
Examples:

- *%s* specifier: It is basically a string specifier for string output.
- $printf "%s\n" "Hello, World!"
  Output:



- *%b specifier*: It is same as string specifier but it allows us to interpret escape sequences with an argument.
- Input: printf "%s\n" "Hello, World! \n" "From Geeks For Geeks\n"
-
- Output: Hello, World! \n
- From Geeks For Geeks\n

Example:
$printf "%b\n" "Geeks" "for" "\nGeeks\n"
Output:



- *%d specifier:* It is an integer specifier for showing the integral values.
- $printf "%d\n" "213" "109"
  Output:

```
hardeep@hardeep-Compaq-Presario-CQ40-Notebook-PC: ~
File  Edit  View  Search  Terminal  Help
hardeep@hardeep-Compaq-Presario-CQ40-Notebook-PC:~$ printf "%d\n" "2
213
109
hardeep@hardeep-Compaq-Presario-CQ40-Notebook-PC:~$
```

- *%f specifier:* It is used for output of floating point values.
- $printf "%f\n" "1.32" "3.14"

Output:

```
hardeep@hardeep-Compaq-Presario-CQ40-Notebook-PC: ~
File  Edit  View  Search  Terminal  Help
hardeep@hardeep-Compaq-Presario-CQ40-Notebook-PC:~$ printf "%f\n" "1
1.320000
3.140000
hardeep@hardeep-Compaq-Presario-CQ40-Notebook-PC:~$
```

script command

script command in Linux is used to make typescript or record all the terminal activities. After executing the *script* command it starts recording everything printed on the screen including the inputs and outputs until exit. By default, all the terminal information is saved in the file *typescript* , if no argument is given. *script* is mostly used when we want to capture the output of a command or a set of command while installing a program or the logs generated on the terminal while compiling an opensource codes, etc. *script* command uses *two files* i.e. one for the terminal output and other for the timing information.

Syntax:

script [options] [file]

Example 1: To start a typescript without any argument. If no filename is given as argument, *script* will automatically create a file namely *typescript* in the home directory to save the recorded information.

Input:

```
sc@ubuntu:~$ script
Script started, file is typescript
sc@ubuntu:~$
```

In order to stop the typescript, we just need to execute *exit* command and script will stop the capturing process. Since there's no filename given as argument, the script will automatically create a file namely *typescript* in the home directory to save the recorded information.

Output:

```
sc@ubuntu:~$ script
Script started, file is typescript
sc@ubuntu:~$ echo "Geeksforgeeks"
Geeksforgeeks
sc@ubuntu:~$ exitScript done, file is typescript
sc@ubuntu:~$
```

passwd Command

passwd - change user password

Syntax

passwd [options] [LOGIN]

The **passwd** command changes passwords for user accounts. A normal user may only change the password for his/her own account, while the superuser may change the password for any account. passwd also changes the account or associated password validity period.

OPTIONS

TAG	DESCRIPTION
-a, --all	This option can be used only with -S and causes show status for all users.
-d, --delete	Delete a user's password (make it empty). This is a quick way to disable a password for an account. It will set the named account passwordless.
-e, --expire	Immediately expire an account's password. This in effect can force a user to change his/her password at the user's next login.
-h, --help	Display help message and exit.
-g, --noheadings	Do not print a header line.
-h, --help	Display help text and exit.

Example-1:
Change your own password:
$ passwd
output:
$ passwd
Changing password for ubuntu.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Example-2:

Change the password for the user named username:
$ sudo passwd username
output:
$ sudo passwd user1
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully

Example-3:
Check the status of the password for the user named user1:
$ sudo passwd -S user1
$ sudo passwd -l user1
output:
passwd: password expiry information changed.

Example-4:

Unlock user1's password. It will automatically be reset to whatever it was before it was locked, and user1 will be able to log in again:
$ sudo passwd -u user1
output:
passwd: password expiry information changed.

Example-5:
Expire user1's password. The next time he logs in, he will be required to set a new password.
$ sudo passwd -e user1
output:
passwd: password expiry information changed.
next time user login:
$ su user1
Password:
You are required to change your password immediately (root enforced)
Changing password for user1.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:

Example-6:
Delete a user's password (make it empty). This is a quick way to disable a password for an account. It will set the named account passwordless.
$ sudo passwd -d user1
output:
passwd: password expiry information changed.

uname command

uname *- prints the name, version and other details about the current machine and the operating system.*

System

uname [OPTION]...

Options

-a, --all
  print all information, in the following order, except omit -p and -i if unknown:
-s, --kernel-name
  print the kernel name
-n, --nodename
  print the network node hostname
--help
  display this help and exit

The **uname** is a command line tool most commonly used to determine the processor architecture, the system hostname and the version of the kernel running on the system.

Examples

When we run uname command without any options, it just prints the core operating system name. We can also use -s option to get the same output, it prints the kernel name of the system.

$ uname
Linux
$ uname -s
$ uname –a

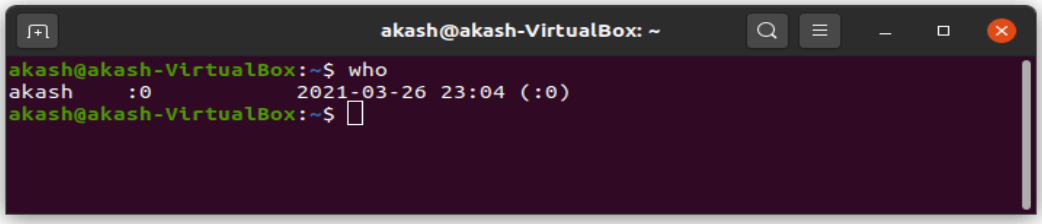
Who command

The Linux "who" command lets you display the users currently logged in to your UNIX or Linux

operating system.

Syntax of the command:

1. who



1. Command to display the hostname and user associated with the input/output devices like Who -m -H

2. To display all details of currently logged in users-

who -a

Or

Syntax

1. Who -all

3. To display information about all active processes that are spawned by the NIT process-

This command will help you to display essential information as well as each and every active process.

Syntax

1. who -p -h

4. To display the status of the user's message as -, + or?

This command will help us to display the status of the user's message. Syntax of this command given below:

Syntax

1. who -T -H

5. To display the whole list of logged-in users-

This command will help us to display the whole list of the logged-in users. Syntax of this command given below:

Syntax

1. who -u

6. To display the whole list of dead processes-

One can use this command to see the complete list of all dead processes. The syntax of the command given below:

Syntax

1. who -d -H

date command

date - print or set the system date and time

Syntax

date [OPTION]... [+FORMAT]
date [-u|--utc|--universal] [MMDDhhmm[[CC]YY][.ss]]

Options

The following are some useful command line options of the date command:

-d, --date=STRING: It is used to display time described by STRING.

-r, --reference=FILE: It is used to display the previous modification time of the FILE.

-s, --set=STRING: It is used to set time described by STRING.

-u, --utc, --universal: It is used to display or set the UTC.

--help: It is used to display the help manual.

--version: It displays the version information.

Formatting options

%a: It is used to display the abbreviated weekday name (e.g., Sun)

%A: It is used to display the full weekday name (e.g., Sunday)

%b: it is used to display the abbreviated month name (e.g., Jan)

%B: It is used to display the full month name (e.g., January)

%c: It is used to display the date and time (e.g., Thu Mar 3 23:05:25 2005)

%C: It is used to display the century; like %Y, except omit last two digits (e.g., 20)

%d: It is used to display the day of the month (e.g., 01)

%D: It is used to display date; same as %m/%d/%y

%e: It is used to display the day of the month, space padded; same as %_d

%F: It is used to display the full date; same as %Y-%m-%d

%g: It is used to display the last two digits of the year of ISO week number (see %G)

%G: It is used to display the year of ISO week number (see %V); normally useful only with %V.

%h: It is similar to the '%b' option.

%H: It is used for an hour (00..23)

%I: It is used for an hour (01..12)

%j: It is used for the day of the year (001..366)

%k: It is used for the hour, space padded (0..23); same as %_H

%l: It is used for the hour, space padded (1..12); same as %_I

%m: It is used for the month (01..12)

%M: It is used for the minute (00..59)

%n: It is used for a new line

%N: it is used for nanoseconds (000000000..999999999)

%p: It is used to pass the equivalent of either AM or PM; blank if not known

%P: It is similar to the '%p' option, But, it is passed in lower case.

%q: It is used to display the quarter of a year (1..4)

%r: It is used to display the 12-hour clock time (e.g., 11:11:04 PM)

%R: It is used to display the 24-hour clock time in an hour and minute; same as %H:%M

%s: it is used to display the seconds since 1970-01-01 00:00:00 UTC

%S: it is used to display the second (00..60)

%t: It is used for a tab option.

%T: it is used to display the time; same as %H:%M:%S

%u: It is used for the day of the week (1..7); 1 is Monday

%U: It is used for a week number of year, it counts Sunday as the first day of the week (00..53)

%Y: It is used for a year

%z: It is used for the +hh:mm numeric time zone (e.g., -0400)

%:z: It is used for the +hh:mm numeric time zone (e.g., -04:00)

%::z: It is used for the +hh:mm:ss numeric time zone (e.g., -04:00:00)

%:::z: It is used for the numeric time zone with ':' to necessary precision (e.g., -04, +05:30)

%Z: It is used for the alphabetic time zone abbreviation (e.g., EDT)

How to use the date Command

The date command is basic utility, and it can be used by executing without any argument. It will display the current date and time. Consider the below command:

1. date

The above command will display the current date and time of the system. Consider the below output:

```
javatpoint@javatpoint-Inspiron-3542:~$ date
Sat Jun  6 20:35:29 IST 2020
javatpoint@javatpoint-Inspiron-3542:~$
```

Date Formatting

We can display the date in the format of our choice. There are several options to display the date in a specific format. Different date formats can be seen from the above list of formatting options. Consider the below commands:

1. date '%a %d-%m-%y'
2. date +'%A %d-%m-%y'
3. date +'%b %d-%m-%y'

4. date +'%B %d-%m-%y'
5. date +'%c %d-%m-%y'
6. date +'%C %d-%m-%y'

From the above commands, the date will display in different formats. Consider the below output:

```
javatpoint@javatpoint-Inspiron-3542:~$ date '%a %d-%m-%y'
date: invalid date ‘%a %d-%m-%y’
javatpoint@javatpoint-Inspiron-3542:~$ date +'%a %d-%m-%y'
Sat 06-06-20
javatpoint@javatpoint-Inspiron-3542:~$ date +'%A %d-%m-%y'
Saturday 06-06-20
javatpoint@javatpoint-Inspiron-3542:~$ date +'%A %y-%m-%d'
Saturday 20-06-06
javatpoint@javatpoint-Inspiron-3542:~$ date +'%b %y-%m-%d'
Jun 20-06-06
javatpoint@javatpoint-Inspiron-3542:~$ date +'%B %y-%m-%d'
June 20-06-06
javatpoint@javatpoint-Inspiron-3542:~$ date +'%c %y-%m-%d'
Saturday 06 June 2020 08:44:58 PM IST 20-06-06
javatpoint@javatpoint-Inspiron-3542:~$ date +'%C %y-%m-%d'
20 20-06-06
```

Similar as the above commands, there are other options available. See the complete list from the above formatting options.

Date Strings

We can specify the date in different date strings. The '-d' option is used to operate on a specific date. Consider the below command:

1. date -d "2020-06-06 12:10:53"

From the above command, we have a specified date in a different format. It will display the description of the specific date and time. Consider the below output:

```
javatpoint@javatpoint-Inspiron-3542:~$ date -d "2020-06-06 12:10:53"
Sat Jun  6 12:10:53 IST 2020
```

We can also use the custom formatting as follows:

1. date -d '06 jun 2020' +'%A, %d %B %Y'

stty command

The **stty** command changes and prints terminal line settings.
This page covers the GNU/Linux version of **stty**.
stty displays or changes the characteristics of the terminal.

Syntax

stty [-F *DEVICE* | --file=*DEVICE*] [*SETTING*]...
stty [-F *DEVICE* | --file=*DEVICE*] [-a|--all]
stty [-F *DEVICE* | --file=*DEVICE*] [-g|--save]

Options

-a, **--all**	Print all current settings in human-readable form.
-g, **--save**	Print all current settings in a **stty**-readable form.
-F, **--file=***DEVICE*	Open and use the specified *DEVICE* instead of stdin.
--help	Display a help message, and exit.
--version	Output version information, and exit.

You can also use –save option, which is same as -g

```
# stty --save
```

```
500:5:bf:8a3b:3:1c:7f:15:4:0:1:0:11:13:1a:0:12:f:17:16:0:0:0:0:0:0:0:0:0:0:0:0:0:0
:0:0
```

Specify Device

You can specify a device file as an argument to stty command. In that case, it will use the device that you've specified instead of using the standard input.

```
# stty -F /dev/pts/0
speed 38400 baud; line = 0;
-brkint -imaxbel
```

pwd Command

The pwd command is used to display the location of the current working directory.

Syntax:

1. pwd

cd Command

The cd command is used to change the current directory.

Syntax:

1. cd <directory name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ cd Desktop
javatpoint@javatpoint-Inspiron-3542:~/Desktop$
```

mkdir Command

The mkdir command is used to create a new directory under any directory.

Syntax:

1. mkdir <directory name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ mkdir new_directory
javatpoint@javatpoint-Inspiron-3542:~$
```

rmdir Command

The rmdir command is used to delete a directory.

Syntax:

1. rmdir <directory name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ rmdir new_directory
javatpoint@javatpoint-Inspiron-3542:~$
```

ls Command

The ls command is used to display a list of content of a directory.

Syntax:

1. ls

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ ls
a               Desktop           examples.desktop  Music         sample
Akash           Directory         hello.c           pico          snap
a.out           Documents         hello.i           Pictures      Templat
composer.phar   Downloads         hello.o           project       Test.tx
Demo.sh         eclipse           hello.s           Public        Videos
Demo.txt        eclipse-installer index.html        Python
Demo.txt~       eclipse-workspace mail              Python-3.8.0
```

cp Command

The cp command is used to copy a file or directory.

Syntax:

To copy in the same directory:

1. cp <existing file name> <new file name>

To copy in a different directory:

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ cp demo.txt demo1.txt
javatpoint@javatpoint-Inspiron-3542:~$ cp demo.txt Documents
```

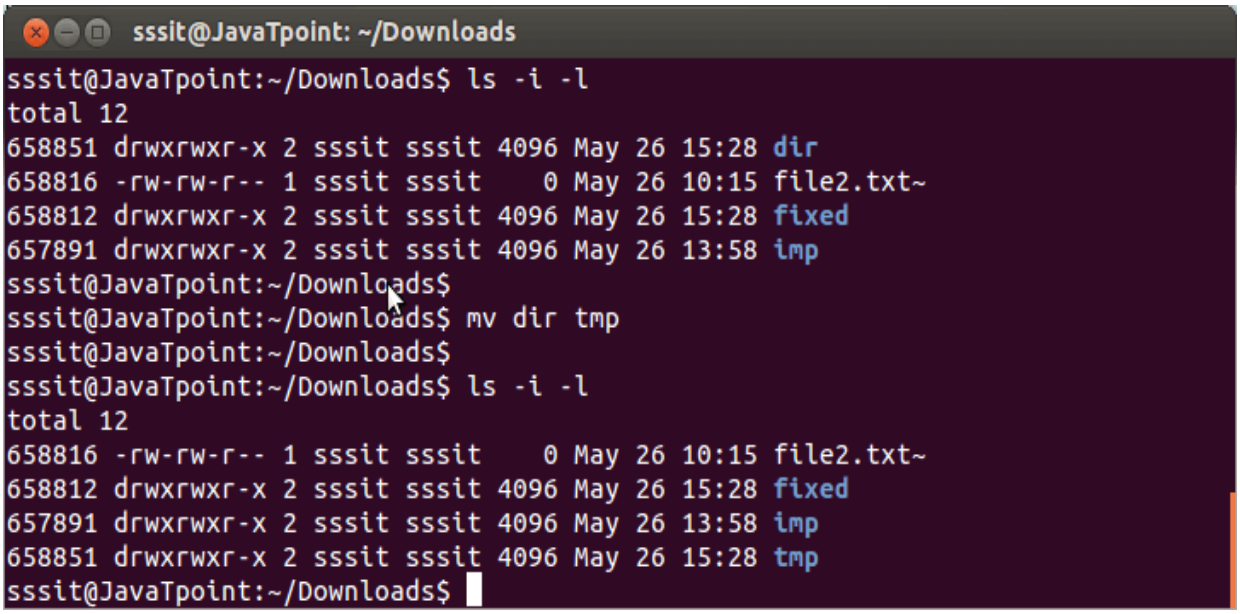
mv Command

The mv command is used to move a file or a directory form one location to another location.

Syntax:

1. mv <file name> <directory path>

Linux mv command is used to move existing file or directory from one location to another. It is also used to rename a file or directory. If you want to rename a single directory or file then **'mv'** option will be better to use.\



mv Option

Option	Function
mv -i	Asks for permission to over write.
mv *	Move multiple files to a specific directory.
mv --suffix	Used to take backup before over writing.
mv -u	Only move those files that doesn't exist.

rm Command

The rm command is used to remove a file.

Syntax:

rm <file name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ rm Demo.txt
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ rm Demo1.txt Demo2.txt
```

touch Command

The touch command is used to create empty files. We can create multiple empty files by executing it once.

Syntax:

1. touch <file name>
2. touch <file1> <file2> ....

Output:

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ touch Demo.txt
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ touch Demo1.txt Demo2.tx
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ ls
Demo1.txt  Demo2.txt  Demo.txt
```

cat Command

The cat command is a multi-purpose utility in the Linux system. It can be used to create a file, display content of the file, copy the content of one file to another file, and more.

Syntax:

1. cat [OPTION]... [FILE]..

To create a file, execute it as follows:

1. cat > <file name>
2. // Enter file content

Press "**CTRL+ D**" keys to save the file. To display the content of the file, execute it as follows:

1. cat <file name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ cat > Demo.txt
This is a text file.
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ cat Demo.txt
This is a text file.
```

more command

As 'cat' command displays the file content. Same way 'more' command also displays the content of a file. Only difference is that, in case of larger files, 'cat' command output will scroll off your screen while 'more' command displays output one screenful at a time.

Following keys are used in 'more' command to scroll the page:

- Enter key: To scroll down page line by line.
- Space bar: To go to next page.
- b key: To go to the backward page.
- / key: Lets you search the string.

Syntax:

1. more <file name>

Example:

1. more /var/log/udev

Look at the above snapshot, in the left corner it shows 0%, which indicates that 0% page is displayed.
To scroll down use **'space'** button, next page will be displayed.
If you want to scroll down the page line by line use **'enter'** key.
If you want to go to the last or backward page use **'b'** key.

Linux more options

Options	Function
more -num	Limits the line displayed per page.
more -d	Displays user message at right corner.
more -s	Squeeze blank lines.
more +/string name	It helps to find the string.
more +num	Used to display the content from a specific line.

Note: The 'more' command can't be used to display binary files.

lines, number of characters, and the number of words in a file. Mostly, it is used with pipes for counting operation.

wc command

The wc command shows one line of counts for all files, and it shows the file name pursuing the counts if the file was provided as an argument. The wc command shows the final line having the cumulative counts along with the 'total' file name if more than a FILE is provided. The counts are displayed in order, i.e., newlines, words, characters, bytes, and maximum line length.

Syntax:

1. wc [OPTION]... [FILE]...
2. wc [OPTION]... --files0-from=F

Options:

Some useful command line options supported by the wc command are as following:

-c, --bytes: It is used to print the byte counts.
-m, --chars: It is used to print the character counts.
-l, --lines: It is used to print the newline counts.
--files0-from=F: It is used to read input from specified files.
-L, --max-line-length: It is used to print the maximum display width.
-w, --words: It is used to print the word counts.
--help: It is used to display the help manual.
--version: It is used to display the version information.

Display count information of a file

To display the complete count information of a file, execute the command without any argument. It will display the number of lines, words, and bytes from the file. Execute the command as follows:

1. wc <file name>

Consider the below command:

1. wc exm.txt

The above command will display the number of lines, number of words, number of bytes, and file name from the file 'exm.txt'. Consider the below output:

```
javatpoint@javatpoint-Inspiron-3542:~$ cat exm.txt
Apple is Red.
mango is yellow.
your dress color is Red,
Red color suits on all.
javatpoint@javatpoint-Inspiron-3542:~$ wc exm.txt
 4 16 80 exm.txt
```

Display count information of multiple files

To display the complete count information of multiple files at once, specify the file names after space (' '). It is executed as follows:

1. wc <file1> <file2>

Consider the below example:

1. wc exm.txt marks.txt

The above command will display the number of words, the number of characters, and the number of the bytes from the files 'exm.txt' and 'marks.txt'. Consider the below output:

```
javatpoint@javatpoint-Inspiron-3542:~$ wc exm.txt marks.txt
  4  16  80 exm.txt
  6   6  52 marks.txt
 10  22 132 total
```

Display the number of lines in a file

The '-l' option is used to display the number of lines in a file. It is executed as follows:

1. wc - l <file name>

```
javatpoint@javatpoint-Inspiron-3542:~$ wc -l exm.txt
4 exm.txt
```

Display the number of characters in a file

The '-m' option is used to display the number of characters in a file. It is executed as follows:

1. wc -m <file name>

Consider the below example:

1. wc -m exm.txt

The above command will display the number of words from the file 'exm.txt'. Consider the below output:

```
javatpoint@javatpoint-Inspiron-3542:~$ wc -m exm.txt
80 exm.txt
```

Linux lp and lpr command

CUPS (Common Unix Printing System) provides the system commands for printing files. Additionally, it supports several standard options to control the print operation. Let's see how to print files.

How to print files in Linux

To print files from Linux terminal, execute the lp and lpr command. It will print the file from the default printer. Execute it as follows:

1. lp < file name>
2. lpr <file name>

The above commands will print the specified file.

Choosing a Printer

If our system has connected with more than one printer, you can choose one of them to print the particular file. These printers can be connected with different methods such as parallel, serial, USB port, networking.

To display a list of the connected printers, execute the command as follows:

1. lpstat -p -d

The above command will display a list of available printers. The '-p' option is used to display a list of the printers, and the '=d' option specifies the default printer.

To print a file with a specific printer, execute the lp command with the '-d' option or lpr command with the '-P' option. Consider the following commands:

1. lp -d <printer name> <file name>
2. lpr -P <printer name> <file name>

In the above commands, both commands will perform the same operation.

For example, to print a file 'Notes.pdf' by the printer name 'HPM375', execute the command as follows:

1. lpr -P HPM375 Notes.pdf

od command

od command in Linux is used to convert the content of input in different formats with octal format as the default format.This command is especially useful when debugging Linux scripts for unwanted changes or characters. If more than one file is specified, od command concatenates them in the listed order to form the input.It can display output in a variety of other formats, including hexadecimal, decimal, and ASCII. It is useful for visualizing data that is not in a human-readable format, like the executable code of a program.

Syntax :

```
od [OPTION]... [FILE]...
```

Options of od command

1. -b Option : It displays the contents of input in octal format.

SYNTAX :

```
$ od -b input.txt
```

EXAMPLE :

```
input :
100
101
102
103
104
105
```

tar command

The Linux 'tar' stands for tape archive, which is used to create Archive and extract the Archive files. tar command in Linux is one of the important commands which provides archiving functionality in Linux. We can use the Linux tar command to create compressed or uncompressed Archive files and also maintain and modify them.

Syntax of `tar` command in Linux

```
tar [options] [archive-file] [file or directory to be archived]
```

Options	Description
-c	Creates an archive by bundling files and directories together.
-x	Extracts files and directories from an existing archive.
-f	Specifies the filename of the archive to be created or extracted.
-t	Displays or lists the files and directories contained within an archive.
-u	Archives and adds new files or directories to an existing archive.
-v	Displays verbose information, providing detailed output during the archiving or extraction process.
-A	Concatenates multiple archive files into a single archive.
-r	Updates or adds files or directories to an already existing archive without recreating the entire archive.

What is an Archive file?

An Archive file is a file that is composed of one or more files along with metadata. Archive files are used to collect multiple data files together into a single file for easier portability and storage, or simply to compress files to use less storage space.

Examples:

1. Creating an uncompressed tar Archive using option -cvf

This command creates a tar file called file.tar which is the Archive of all .c files in the current directory.

This command extracts files from Archives.

```
tar xvf file.tar
```

- '-x': Extracts files from an archive.

- ‘-v’: Displays verbose output during the extraction process.
- ‘-f’: Specifies the filename of the archive.

This command creates a tar file called file.tar.gz which is the Archive of .c files.

```
tar cvzf file.tar.gz *.c
```

- ‘-z’: Uses gzip compression.
- ‘-j’: Uses bzip2 compression.
- ‘-J’: Uses xz compression.

This command extracts files from tar archived file.tar.gz files.

```
tar xvzf file.tar.gz
```

This command compresses and creates archive files less than the size of the gzip. Both compress and decompress take more time than gzip.

```
tar cvfj file.tar.tbz example.cpp
```

gzip command

gzip command compresses files. Each single file is compressed into a single file. The compressed file consists of a GNU zip header and deflated data.

If given a file as an argument, gzip compresses the file, adds a “.gz” suffix, and deletes the original file. With no arguments, gzip compresses the standard input and writes the compressed file to standard output.

Syntax :

```
gzip [Options] [filenames]
```

Example:

```
$ gzip mydoc.txt
```

This command will create a compressed file of mydoc.txt named as mydoc.txt.gz and delete the original file.

Unlink command

In Linux, the “**unlink**” is a powerful command utility that enables one to remove a file or link based on its specific name. This command can also remove a symbolic link or the soft link. The unlink command is an alternative to the “**rm**” command for removing files.

General Syntax

```
$ unlink linkname/filename
```

In the above syntax, the **unlink** command removes the single file or link from the system specifying the name.

You can get a more detailed usage of the unlink command using the command:

```
$ unlink --help
```

Remove File

The “**unlink**” command is utilized here for removing the “**file.c**” from the directory.

```
$ unlink file.c
```

The above command has successfully removed the “**file.c**” from the system.

```
$ ls
```

The output shows that the “**file.c**” file has been successfully removed from the system.

du command

du command is used for estimating file and directory space usage. The name `du` stands for “disk usage”. It provides information about the storage consumption of files and directories.

Syntax of `du` command in Linux

```
du [options] [directory/file]
```

Here,

[options] = various flags that modify the behavior of the command

[directory/file] = the target directory or file for which disk usage information is desired. If no

directory or file is specified, du will default to the current directory.

Example:

```
du /home/mandeep/test
```

Output:

```
44    /home/mandeep/test/data
2012    /home/mandeep/test/system design
24    /home/mandeep/test/table/sample_table/tree
28    /home/mandeep/test/table/sample_table
32    /home/mandeep/test/table
100104    /home/mandeep/test
```

Options available in `du` command in Linux

The `du` command offers a range of options that enhance its functionality. Let's explore some commonly used options:

Options	Describe
-0, –null	end each output line with NULL
-a or –all	Displays disk usage information for all files and directories, including hidden ones.
–apparent-size	print apparent sizes, rather than disk usage
-B, –block-size=SIZE	scale sizes to SIZE before printing on console

`-h` option in `du` command

If we want to print sizes in human readable format(K, M, G), use -h option

```
du -h /home/Mandeep/test
```

To calculate the total size of a directory and its subdirectories:

Use **-c** option to print total size

```
du -c -h /home/mandeep/test
```

`-d` option in `du` command

To print sizes to particular level, use **-d** option with level no.

```
du -d 1 /home/mandeep/test
```

Output:

```
44    /home/mandeep/test/data
2012    /home/mandeep/test/system design
32    /home/mandeep/test/table
100104    /home/mandeep/test
```

Now try with level 2, you will get some extra directories

```
du -d 2 /home/mandeep/test
```

Output:

```
44    /home/mandeep/test/data
2012    /home/mandeep/test/system design
28    /home/mandeep/test/table/sample_table
32    /home/mandeep/test/table
100104    /home/mandeep/test
```

To obtain the disk usage summary for a directory:

Get summary of file system using **-s** option

```
du -s /home/mandeep/test
```

Output:

```
100104    /home/mandeep/test
```

df command

disk free also known as `df`, which is a powerful utility that provides valuable information on disk

space utilization. The **df** command displays information about file system disk space usage on the mounted file system. This command retrieves the information from `/proc/mounts` or `/etc/mtab`. By default, df command shows disk space in Kilobytes (KB) and uses the SI unit suffixes (e.g, M for megabytes, G for gigabytes) for clarity.

Syntax of `df`

```
df [OPTION]... [FILE]...
```

If no file name is given, it displays the space available on all currently mounted file systems.

For example:

```
df
```

This will display information about all the mounted file systems which will include total size, used space, usage percentage, and the mount point.

df

Now, if you specify a particular file, then it will show the mount information of that particular file.

For example:

```
df jayesh.txt
```

df jayesh.txt

You can replace `jayesh.txt` with the desired file name

Options Available in `df` command in Linux

Options	Description
'-a' or '–all'	Includes pseudo, duplicate, and inaccessible file systems in the output.
'-B <SIZE>' or '–block-size=<SIZE>'	Scales sizes by SIZE before printing them.
'-h' or '–human-readable'	Prints sizes in a human-readable format using power of 1024.
'-H' or '–si'	Prints sizes in a human-readable format using power of 1000.
'-i' or '–inodes'	Lists inode information instead of block usage.
'-l' or '–local'	Limits listing to local file systems.
'-T' or '–print-type'	Prints file system type

`-a` option in `df` command in Linux

If you want to display all the file system, use -a option.

```
df -a
```

df -a

`-h` or `-H` option in `df` command in Linux

Use -h option to display size in power of 1024

```
df -h jayesh.txt
```

df -h jayesh.txt

Use -H option to display sizes in power of 1000

```
df -H jayesh.txt
```

df -H jayesh.txt

`–total` option in `df` command in Linux

To get complete grand total, use –total option

```
df --total
```

df –total

`-T` option in `df` command in Linux.

Use -T option to display file type

For example:

```
df -T jayesh.txt
```

df -T jayesh.txt
You can see the file type for `jayesh.txt` is ext4
`–help` option in `df` command in Linux
And for more help, you can use –help option.

```
df --help
```

mount command.
mount command is used to mount the filesystem found on a device to big tree structure(**Linux** filesystem) rooted at '/'. Conversely, another command **umount** can be used to detach these devices from the Tree.
Syntax:

```
mount -t type device dir
```

Note: It is important to note that we are only discussing the standard form of **mount** command given as syntax. Different forms are somewhat discussed because it has certain limitations on different kernels.
Some Important Options:

- **l** : Lists all the file systems mounted yet.
- **h** : Displays options for command.
- **V** : Displays the version information.
- **a** : Mounts all devices described at **/etc/fstab**.
- **t** : Type of filesystem device uses.
- **T** : Describes an alternative fstab file.
- **r** : Read-only mode mounted.

Examples:
umount - unmount file systems

SYNTAX
umount [-hV]
umount -a [-dflnrv] [-t *vfstype***] [-O** *options***]**

The **umount** command detaches the file system(s) mentioned from the file hierarchy. A file system is specified by giving the directory where it has been mounted. Giving the special device on which the file system lives may also work, but is obsolete, mainly because it will fail in case this device was mounted on more than one directory.

Options for the **umount** command:

Tag	Description
-V	Print version and exit.
-h	Print help message and exit.
-v	Verbose mode.
-n	Unmount without writing in */etc/mtab*.
-r	In case unmounting fails, try to remount read-only.
-d	In case the unmounted device was a loop device, also free this loop device.

find command

The find command helps us to find a particular file within a directory. It is used to find the list of files

for the various conditions like permission, user ownership, modification, date/time, size, and more. The find utility comes by default with most of the Linux distros, so we don't need to install any additional package. It is one of the most essential and used commands of the Linux system.

find <location> <comparison-criteria> <search-term>

The following symbols are used to specify the directory:
(.) : For current directory name
(/) : For the root directory

Find files by name

We can search all the files ending with the extension '.txt.' To do so, execute the below command:

```
find . -name "*.txt"
```

Finding files by type

The '-type' parameter is used to specify the file type.
Some of the file types are as follows:

- f: regular file
- d: directory
- l: symbolic links
- c: character devices
- b: block devices

Consider the below command:

1. find . -type d -name "*.bak"

The above command will list all the directories having '.bak' extension. Consider the below output: From the above output, the command "find . -type d -name "*.bak" is displaying all the directories ending with '.bak'.

Find newer files

The '-newer' parameter helps in searching the files which are newer than the mentioned file. Consider the below command:

```
find . -newer msg.txt
```

umask command

The user file creation mode mask (umask) is a built-in shell command that may be used to set default values for the read/write/execution permissions on newly created files. It should be executed in either the ".cshrc" or ".profile" shell startup files.
It is given a three-digit octal value, which represents the binary inverse of the permissions which may be assigned to files. This three-digit numeric argument represents the access to be "inhibited" or "masked out" when a file is created.

Octal	Binary	Permissions
0	000	rwx
1	001	rw-
2	010	r-x
3	011	r–
4	100	-wx
5	101	-w-
6	110	–x
7	111	(none)

Changing the "umask"

To determine the desired value, simply figure out the numeric equivalent for the file mode you want to get and then subtract it from 777.

For example, to obtain the mode 751 by default, compute 777-751=026 – this is the value you give to "umask"

```
% umask 026
```

Once this command is executed, all future files created will be given this protection automatically. Another to calculate "umask" values is to remember that the number 2 in the "umask" turns off write permission, while the number 7 turns off read, write and execute permission.

Common "unmask" Values

By default, most UNIX versions specify an octal mode of 666 (any user can read or write the file) when they create new files. Likewise, new programs are created with a mode of 777 (any user can read, write, or execute the program).

The most common umask values are 022, 027, and 077. A umask value of 022 lets the owner both read and write all newly created files, but everybody else can only read them:

- **0666**: Default file creation mode.
- **022**: resultant mode.
- **0644**: resultant mode.

Ulimit command

ulimit is admin access required Linux shell command which is used to see, set, or limit the resource usage of the current user. It is used to return the number of open file descriptors for each process. It is also used to set restrictions on the resources used by a process.

Syntax:

To check the ulimit value use the following command:

```
ulimit -a
```

Working with ulimit commands:

To display maximum users process or for showing maximum user process limit for the logged-in user.

```
ulimit -u
```

For showing the maximum file size a user can have.

```
ulimit -f
```

For showing maximum memory size for the current user.

```
ulimit -m
```

For showing maximum memory size limit.

```
ulimit -v
```

What are Soft limits and Hard limits in Linux?

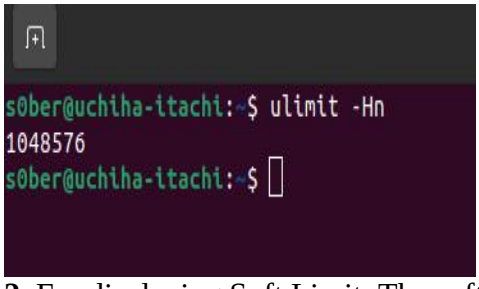
The soft limits are the limits which are allocated for actual processing of application or users while the Hard limits are nothing but an upper bound to the values of soft limits. Hence,

```
(soft limits <= hard limit)
```

Working with Hard and Soft limit values:

For displaying the Hard limit. Hard limits are a restriction to the maximum value of soft limits

```
ulimit -Hn
```



2. For displaying Soft Limit. The soft limits are the limits that are there for processing.

```
ulimit -Sn
```

To change Soft Limit values:

```
sysctl -w fs.file-max=<value>
```

Note: Replace <value> with the value you want to set for the soft limit and also remember size can not exceed the Hard Limit!
Displaying current Values for opened files

```
cat /proc/sys/fs/file-max
```

ps command

Linux provides us a utility called **ps** for viewing information related with the processes on a system which stands as abbreviation for **"Process Status".** ps command is used to list the currently running processes and their PIDs along with some other information depends on different options. It reads the process information from the virtual files in **/proc** file-system. /proc contains virtual files, this is the reason it's referred as a virtual file system.
ps provides numerous options for manipulating the output according to our need.
Syntax

```
ps [options]
```

Options for ps Command :

Simple process selection : Shows the processes for the current shell –

```
[root@rhel7 ~]# ps
  PID TTY          TIME CMD
12330 pts/0    00:00:00 bash
21621 pts/0    00:00:00 ps
```

Result contains four columns of information.
Where,
PID – the unique process ID
TTY – terminal type that the user is logged into
TIME – amount of CPU in minutes and seconds that the process has been running
CMD – name of the command that launched the process.
Note – Sometimes when we execute **ps** command, it shows TIME as 00:00:00. It is nothing but the total accumulated CPU utilization time for any process and 00:00:00 indicates no CPU time has been given by the kernel till now. In above example we found that, for bash no CPU time has been given. This is because bash is just a parent process for different processes which needs bash for their execution and bash itself is not utilizing any CPU time till now.

View Processes : View all the running processes use either of the following option with ps –

```
[root@rhel7 ~]# ps -A
[root@rhel7 ~]# ps -e
```

View Processes not associated with a terminal : View all processes except both session leaders and processes not associated with a terminal.

```
[root@rhel7 ~]# ps -a
  PID TTY          TIME CMD
27011 pts/0    00:00:00 man
27016 pts/0    00:00:00 less
27499 pts/1    00:00:00 ps
```

'w' command

The **'w'** command in Linux gives us important information about who is currently using the computer, how much the computer is being used, and what programs are running. It's a handy tool for people who take care of computer systems, as it helps them keep an eye on what users are doing, how much of the computer's power is being used, and how to make everything run smoothly.

Syntax of `w` command in Linux

```
w [options] user [...]
```

```
algoscale@algoscale-Lenovo-ideapad-330-15IKB:~$ w
 16:12:18 up  4:41,  1 user,  load average: 2.48, 2.05, 1.83
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
algoscal tty7     :0               11:31    4:40m  6:53   0.23s /sbin/ups
algoscale@algoscale-Lenovo-ideapad-330-15IKB:~$ w -u
 16:12:31 up  4:41,  1 user,  load average: 2.25, 2.02, 1.82
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
algoscal tty7     :0               11:31    4:40m  6:53   0.23s /sbin/ups
```

Finger command

Finger command is a user information lookup command which gives details of all the users logged in. This tool is generally used by system administrators. It provides details like login name, user name, idle time, login time, and in some cases their email address even. This tool is similar to the Pinky tool but the Pinky tool is just the lightweight version of this tool.
To install *finger* tool use the following commands as per your Linux distribution.

In case of Debian/Ubuntu

```
$sudo apt-get install finger
```

In case of CentOS/RedHat

```
$sudo yum install finger
```

To get idle status and login details of a user.

```
$finger -s manav
```

arp command

arp command manipulates the System's ARP cache. It also allows a complete dump of the ARP cache. ARP stands for Address Resolution Protocol. The primary function of this protocol is to resolve the IP address of a system to its mac address, and hence it works between level 2(Data link layer) and level 3(Network layer).

Syntax:

```
arp [-v] [-i if] [-H type] -a [hostname]
```

Example: Here we created two machines with name *machine1* and *machine2* with IP address 10.0.2.4 and 10.0.2.5

- Screenshot of hosts before adding

```
root@kali:~# more /etc/hosts
127.0.0.1       localhost
127.0.1.1       kali
```

ftp and sftp

The ftp stands for **F**ile **T**ransfer **P**rotocol. It connect to the remote host to exchange files and directories from one host to another over a network which can be LAN or any other.
The sftp stands for **S**ecure **ftp.**
Most of the ftp commands are applicable to sftp. So wherever, you need to use sftp, you can use it at the place of ftp.

---

ftp prompt

The ftp prompt can be used to perform different ftp functions with ftp commands.

Syntax:

1. ftp



Look at the above snapshot, by passing ftp command we'll be directed into the ftp prompt. Here, we can write ftp commands to perform different functions.

---

ftp help

To open ftp command or help page from ftp prompt , **?** Sign or **help** command is used.

Syntax:

1. ?
2. OR
3. help

telnet command

In Linux, the **telnet** command is used to create a remote connection with a system over a TCP/IP network. It allows us to administrate other systems by the terminal. We can run a program to conduct administration.

telnet hostname/IP address

Commands in telnet

Some important telnet commands are listed and explained below. Only enough of all commands to uniquely identify it require to be typed.

- **auth argument[...]:** This command can manipulate the details sent by the option, i.e., ***TELNET AUTHENTICATION***. For this command, some valid arguments are described below:
  - **disable type:** It disables the named authentication type. We can use the command, i.e., ***auth disable***, to get the available type list.
  - **enable type:** It enables the named authentication type. We can use the command, i.e., ***auth enable***, to get the available type list.
  - **status:** It specifies the current status of various authentication types.
- **close:** It closes the TELNET session and comes back to command mode.
- **display argument[...]:** It displays some or all toggle and set values.
- **encrypt argument[...]:** This command can manipulate the details sent by the option, i.e., ***TELNET ENCRYPT***. Some valid arguments for this command are described below:
  - **disable type [input|output]:** It disables the named encryption type. If we omit output and input, both are disabled. We can use the command, i.e., ***encrypt disable***, to get the available type list.
  - **enable type [input|output]:** It enables the named encryption type. If we omit output and input, both are enabled. We can use the command, i.e., ***encrypt enable***, to get the available type list.
  - **input:** It is similar to the command encrypt start input.
  - **-input:** It is similar to the command encrypt stop input.
  - **output:** It is similar to the command encrypt start output.
  - **-output:** It is similar to the command encrypt stop output.
  - **status:** It specifies the current encryption status.

Install Telnet on Linux (Ubuntu)

Installing telnet on Linux is a straight forward process. We can install it by executing the following commands:
Update the Linux system by executing the below command:

1. sudo apt update

The above command will prompt for the user password. Type the password and press **ENTER** key; it will start a daemon process and take a while to update your system.
To install the telnet, execute the below command:

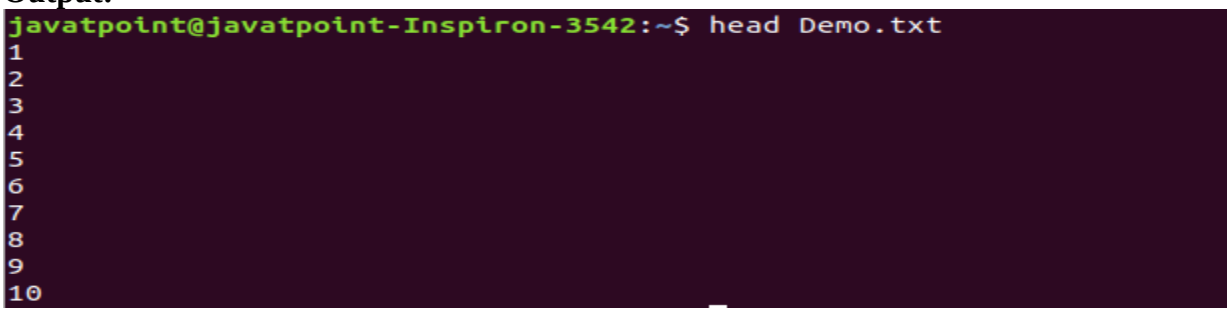
1. sudo apt install telnetd -y

The above command will install the required package for the telnet protocol. Consider the below output:

head Command
The head command is used to display the content of a file. It displays the first 10 lines of a file.
Syntax:

1. head <file name>

Output:



13. tail Command
The tail command is similar to the head command. The difference between both commands is that it displays the last ten lines of the file content. It is useful for reading the error message.
Syntax:

1. tail <file name>

Output:



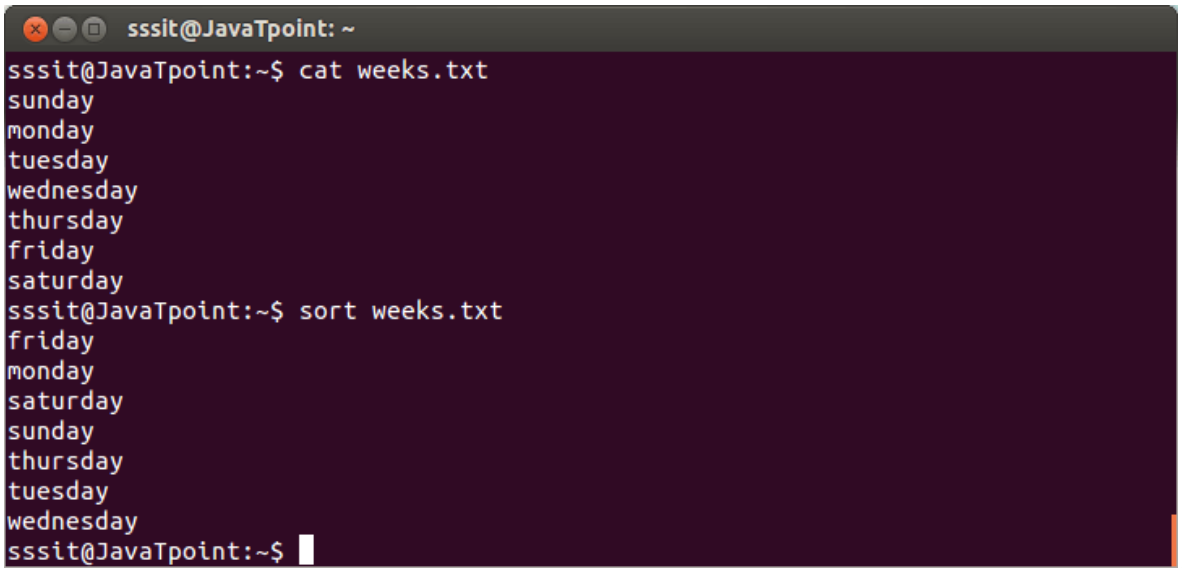
Sort command

The 'sort' command sorts the file content in an alphabetical order.
Syntax:

1. sort <fileName>

Example:

1. sort weeks.txt



nl Command

Numbers lines in a file.

Syntax

nl [**-b** *Type*] [**-f** *Type*] [**-h** *Type*] [**-l** *Number*] [**-d** *Delimiter*] [**-i** *Number*] [**-n** *Format*] [**-v** *Number*] [**-w** *Number*] [**-p**] [**-s** *Separator*] [*File*]

The **nl** command reads the *File* parameter (standard input by default), numbers the lines in the input, and writes the numbered lines to standard output. In the output, the **nl** command numbers the lines on the left according to the flags you specify on the command line.

uniq Command

Linux uniq command is used to remove all the repeated lines from a file. Also, it can be used to display a count of any word, only repeated lines, ignore characters, and compare specific fields. It is one of the most frequently used commands in the Linux system. It is often used with the sort command because it compares adjacent characters. It discards all the identical lines and writes the output.

Syntax:

uniq [OPTION]... [INPUT [OUTPUT]]

Options:

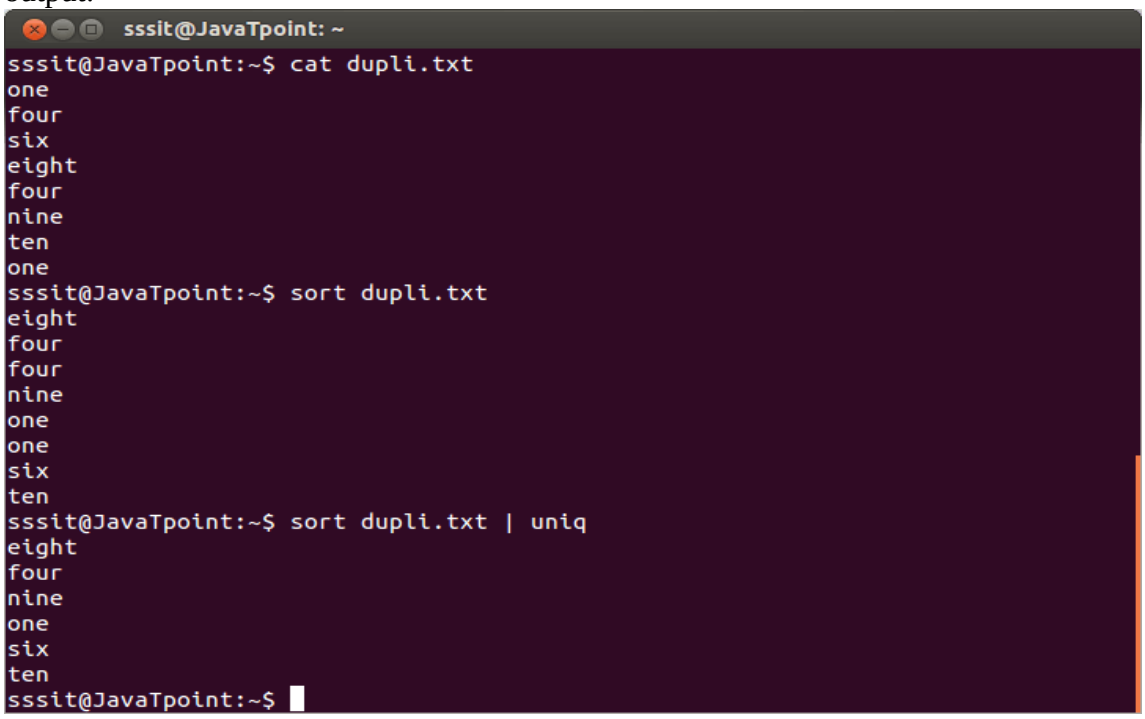
Some useful command line options of the uniq command are as following:

-c, --count: it prefixes the lines by the number of occurrences.
-d, --repeated: it is used to print duplicate lines, one for each group.
-D: It is used to print all the duplicate lines.
--all-repeated[=METHOD]: It is quite similar to the '-D' option, the difference between both the options is that it allows separation of groups with an empty line.
-f, --skip-fields=N: It is used to avoid comparison of the first N fields.
--group[=METHOD]: It is used to display all items and separates the groups with an empty line.
-i, --ignore-case: It is used to ignore the differences while comparing.
-s, --skip-chars=N: It is used to avoid the comparison of the first N characters.
-u, --unique: it is used to print unique lines.
-z, --zero-terminated: It is used for the line delimiter is NUL and not newline mode.
-w, --check-chars=N: It is used to compare not more than N characters in lines.
--help: It is used to display help documentation.
--version: It is used to display the version information.

To remove repeated lines from a file, execute the basic uniq command as follows:

1. sort dupli.txt | uniq

The above command will remove the duplicate lines from the file 'dupli.txt.' Consider the below output:



From the above output, the repeating words are ignored.

Count the number of occurrences of a word

We can count the number of occurrences of a word by using the uniq command. The '-c' option is used to count the word. Execute it as follows:

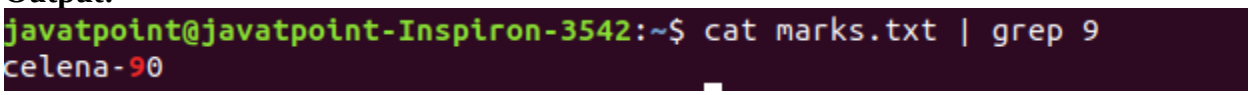
grep Command

The grep is the most powerful and used filter in a Linux system. The 'grep' stands for "**global regular expression print**." It is useful for searching the content from a file. Generally, it is used with the pipe.

Syntax:

1. command | grep <searchWord>

Output:

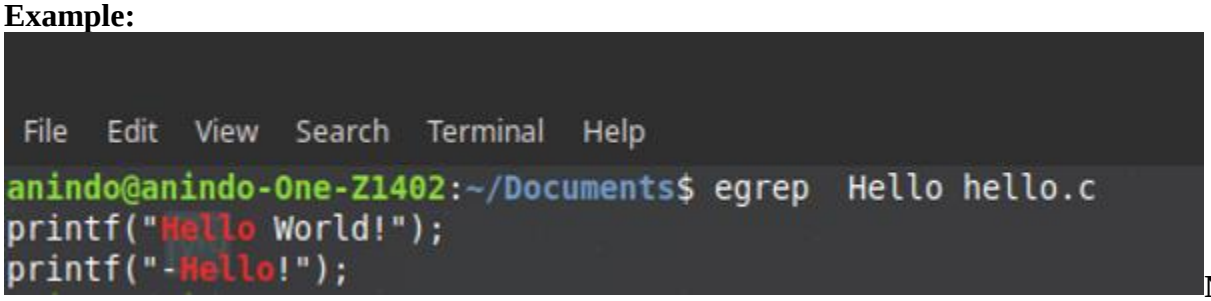


egrep command

egrep **is a pattern searching command which belongs to the family of grep functions**. It works the same way as ***grep -E*** does. It treats the pattern as an extended regular expression and prints out the lines that match the pattern. If there are several files with the matching pattern, it also displays the file names for each line. **Syntax:**

egrep [options] 'PATTERN' files

Example:



Note:

The *egrep* command used mainly due to the fact that it is faster than the grep command. The egrep command treats the meta-characters as they are and do not require to be escaped as is the case with

grep. This allows reducing the overhead of replacing these characters while pattern matching making egrep faster than *grep* or *fgrep*. **Options:** Most of the options for this command are same as ***grep***.

- **-c:** Used to counts and prints the number of lines that matched the pattern and not the lines.

```
anindo@anindo-One-Z1402:~/Documents$ egrep -c Hello hello.c
2
```

- **-v:** It prints the lines that does not match with the pattern.

```
anindo@anindo-One-Z1402:~/Documents$ egrep -v Hello hello.c
#include "stdio.h"

int main(){
return 0;
}
```

- **-i:** Ignore the case of the pattern while matching.

```
anindo@anindo-One-Z1402:~/Documents$ egrep -i hello hello.c
printf("Hello World!");
printf("-Hello!");
```

- **-l**

```
anindo@anindo-One-Z1402:~/Documents$ egrep -w Hello hello.c
printf("Hello World!");
printf("-Hello!");
```

- **-x:** Prints only those lines that matches an entire line of the file.

```
anindo@anindo-One-Z1402:~/Documents$ egrep -x 'return 0;' hello.c
return 0;
```

- **-m NUMBER:** Continue to search for matches till the count reaches NUMBER mentioned as argument.

```
anindo@anindo-One-Z1402:~/Documents$ egrep -m 1 Hello hello.c
printf("Hello World!");
```

- **-o:** Prints only the matched parts of the line and not the entire line for each match.

```
anindo@anindo-One-Z1402:~/Documents$ egrep -o Hello hello.c
Hello
Hello
```

- **-n:** Prints each matched line along with the respective line numbers. For multiple files, prints the file names along with line numbers.
- **-r:** Recursively search for the pattern in all the files of the directory. The last argument is the directory to check. '.' (dot) represents the current directory.

fgrep command in Linux with examples

The *fgrep* filter is used to search for the fixed-character strings in a file. There can be multiple files also to be searched. This command is useful when you need to search for strings which contain lots of regular expression metacharacters, such as "^", "$", etc.

Syntax:

fgrep [options] [-e pattern_list] [pattern] [file]

Options with Description:

- **-c :** It is used to print only a count of the lines which contain the pattern.
- **-h :** Used to display the matched lines.
- **-i :** During comparisons, it will ignore upper/lower case distinction.
- **-l :** Used to print the names of files with matching lines once, separated by new-lines. It will not repeat the names of files when the pattern is found more than once.
- **-n :** It is used precede each line by its line number in the file (first line is 1).

- **-s :** It will only display the error messages.
- **-v :** Print all lines except those contain the pattern.
- **-x :** Print only lines matched entirely.
- **-e pattern_list :** Search for a string in pattern-list (useful when the string begins with a "-").
- **-f pattern-file :** Take the list of patterns from pattern-file.
- **pattern :** Specify a pattern to be used during the search for input.
- **file :** A path name of a file to be searched for the patterns. If no file operands are specified, the standard input will be used.

Example:

$fgrep -c "usin.g" para

```
paras@paras:~/Desktop/coding/my-project/GFG articles/frep$ cat para
Hi,@re you usin.g geeks*forgeeks for learni\ng computer science con/cepts
Geeks*forgeeks is best for learni\ng.
paras@paras:~/Desktop/coding/my-project/GFG articles/frep$ fgrep -c "usin
1
paras@paras:~/Desktop/coding/my-project/GFG articles/frep$
```

-h option: To display the matched lines.

Example:

fgrep -h "usin.g" para

-i option: Used in case insensitive search. It ignore upper/lower case distinction during comparisons. It matches words like : "geeks*forgeeks", "Geeks*forgeeks".

Example:

fgrep -i "geeks*forgeeks" para

cut Command

The cut command is used to select a specific column of a file. The '-d' option is used as a delimiter, and it can be a space (' '), a slash (/), a hyphen (-), or anything else. And, the '-f' option is used to specify a column number.

Syntax:

1. cut -d(delimiter) -f(columnNumber) <fileName>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ cat >marks.txt
alex-50
alen-70
jon-75
carry-85
celena-90
justin-80
javatpoint@javatpoint-Inspiron-3542:~$ cut -d- -f2 marks.txt
50
70
75
85
90
80
javatpoint@javatpoint-Inspiron-3542:~$
```

Paste command

Paste command is one of the useful commands in Unix or Linux operating system. It is used to join files horizontally (parallel merging) by outputting lines consisting of lines from each file specified, separated by **tab** as delimiter, to the standard output. When no file is specified, or put dash ("-") instead of file name, paste reads from standard input and gives output as it is until a interrupt command **[Ctrl-c]** is given. **Syntax:**

paste [OPTION]... [FILES]...

$ cat state

Arunachal Pradesh
Assam
Andhra Pradesh
Bihar
Chhattisgrah

$ cat capital

Itanagar
Dispur
Hyderabad
Patna
Raipur

Without any option paste merges the files in parallel. The paste command writes corresponding lines from the files with tab as a deliminator on the terminal.

$ paste number state capital

1 Arunachal Pradesh Itanagar
2 Assam Dispur
3 Andhra Pradesh Hyderabad
4 Bihar Patna
5 Chhattisgrah Raipur

More than one character is specified

$ paste -d "|," number state capital

1|Arunachal Pradesh,Itanagar
2|Assam,Dispur
3|Andhra Pradesh,Hyderabad
4|Bihar,Patna
5|Chhattisgrah,Raipur

join command

The join command in UNIX is a command line utility for joining lines of two files on a common field. Suppose you have two files and there is a need to combine these two files in a way that the output makes even more sense.

Syntax:

```
$join [OPTION] FILE1 FILE2
```

Example : Let us assume there are two files file1.txt and file2.txt and we want to combine the contents of these two files.

```
// displaying the contents of first file //
$cat file1.txt
1 AAYUSH
2 APAAR
3 HEMANT
4 KARTIK

// displaying contents of second file //
$cat file2.txt
1 101
2 102
3 103
4 104
```

Now, in order to combine two files the files must have some common field. In this case, we have the numbering 1, 2... as the common field in both the files.

NOTE : When using join command, both the input files should be sorted on the KEY on which we are going to join the files.

```
//..using join command.../
$join file1.txt file2.txt
1 AAYUSH 101
2 APAAR 102
3 HEMANT 103
4 KARTIK 104
```

tee command

tee command reads the standard input and writes it to both the standard output and one or more files. The command is named after the T-splitter used in plumbing. It basically breaks the output of a program so that it can be both displayed and saved in a file. It does both the tasks simultaneously, copies the result into the specified files or variables and also display the result.

SYNTAX:

```
tee [OPTION]... [FILE]...
```

Options :

1.-a Option : It basically do not overwrite the file but append to the given file.

Suppose we have **file1.txt**

```
Input: geek
       for
       geeks
```

and **file2.txt**

```
Input:geeks
      for
      geeks
```

SYNTAX :

```
geek@HP:~$ wc -l file1.txt|tee -a file2.txt
```

OUTPUT :

```
3 file1.txt
geek@HP:~$cat file2.txt
OUTPUT:
```

```
geeks
for
geeks
3 file1.txt
```

pg command

use the pg command to read the files named in the File parameter and writes them to standard output one screen at a time.

If you specify hyphen (-) as the File parameter or run the pg command without options, the pg command reads standard input. Each screen is followed by a prompt. If you press the Enter key, another screen displays. Subcommands used with the pg command let you review content that has already passed.

For example, to look at the contents of the file myfile one page at a time, type the following:

```
pg myfile
```

comm command

comm compare two sorted files line by line and write to standard output; the lines that are common and the lines that are unique.

Suppose you have two lists of people and you are asked to find out the names available in one and not in the other, or even those common to both. **comm** is the command that will help you to achieve this. It requires two sorted files which it compares line by line.

Before discussing anything further first let's check out the syntax of **comm** command:

Syntax :

```
$comm [OPTION]... FILE1 FILE2
```

- As using comm, we are trying to compare two files therefore the syntax of comm command needs two filenames as arguments.
- With no OPTION used, comm produces three-column output where first column contains lines unique to FILE1 ,second column contains lines unique to FILE2 and third and last column contains lines common to both the files.
- comm command only works right if you are comparing two files which are **already sorted**.

  Example: Let us suppose there are two sorted files file1.txt and file2.txt and now we will use comm command to compare these two.

```
// displaying contents of file1 //
$cat file1.txt
Apaar
Ayush Rajput
Deepak
Hemant

// displaying contents of file2 //
$cat file2.txt
Apaar
Hemant
Lucky
Pranjal Thakral
```

  Now, run comm command as:

```
// using comm command for
comparing two files //
$comm file1.txt file2.txt
                Apaar
```

cmp command

cmp command in Linux/UNIX is used to compare the two files byte by byte and helps you to find out whether the two files are identical or not.

- When cmp is used for comparison between two files, it reports the location of the first mismatch to the screen if difference is found and if no difference is found *i.e* the files compared are identical.

- cmp displays no message and simply returns the prompt if the files compared are identical.

```
Syntax:
cmp [OPTION]... FILE1 [FILE2 [SKIP1 [SKIP2]]]

$cmp file1.txt file2.txt
```

If the files are not identical : the output of the above command will be :

```
$cmp file1.txt file2.txt
file1.txt file2.txt differ: byte 9, line 2
```

diff command

diff stands for **difference**. This command is used to display the differences in the files by comparing the files line by line. Unlike its fellow members, cmp and comm, it tells us which lines in one file have is to be changed to make the two files identical.

The important thing to remember is that **diff** uses certain **special symbols** and **instructions** that are required to make two files identical. It tells you the instructions on how to change the first file to make it match the second file.

Special symbols are:

```
a : add
c : change
d : delete
```

Syntax :

```
diff [options] File1 File2
```

Lets say we have two files with names **a.txt** and **b.txt** containing 5 Indian states.

```
$ ls
a.txt  b.txt

$ cat a.txt
Gujarat
Uttar Pradesh
Kolkata
Bihar
Jammu and Kashmir
```

tr Command

The tr command is used to translate the file content like from lower case to upper case.

Syntax:

1. command | tr <'old'> <'new'>

Output:

cpio command

cpio stands for "**copy in, copy out**". It is used for processing the archive files like **.cpio* or **.tar*. This command can copy files to and from archives.

Synopsis:

- **Copy-out Mode:** Copy files named in name-list to the archive
  Syntax:
  `cpio -o < name-list > archive`
- **Copy-in Mode:** Extract files from the archive
  Syntax:
  `cpio -i < archive`